Programmeeropdracht Python OOP – Bibliotheek- en uitleensysteem

Context

Je ontwikkelt een **bibliotheek- en uitleensysteem** dat informatie bijhoudt over materialen (boeken, tijdschriften, dvd's...), leden en uitleningen.
De nadruk ligt op **een goede OOP-structuur in Python**.

Belangrijk:

- Gebruik **geen** database, geen bestandsopslag en geen grafische (GUI) interface.
- Een eenvoudige **CLI / tekstinterface** (menu, input() in main.py) is **toegestaan maar niet verplicht**.
- Werk in alle gevallen met Python-klassen en in-memory datastructuren.

Doel van het systeem

Het systeem moet:

- Materialen in de bibliotheek beheren (boeken, tijdschriften, dvd's, ...).
- Leden (leden van de bibliotheek) beheren.
- Uitleningen registreren en controleren of iets nog beschikbaar is.
- Overzicht geven van welke materialen uitgeleend zijn, wie ze bezit en of er achterstanden (uitleningen die te laat zijn teruggebracht) zijn.

Je toont hiermee aan dat je **OOP-technieken in Python** op een gestructureerde manier kan toepassen.

Vereiste OOP-concepten

In je uitwerking moeten minstens de volgende zaken **zichtbaar en zinvol** gebruikt worden:

1. Klassen & objecten
2. **Inheritance** (erfelijkheid)
3. **Composition** (objecten die andere objecten bevatten)
4. **Custom exceptions & error handling**
5. **Magic methods** (zoals __str__, __repr__, __len__, ...)
6. **Scope & modules** (meerdere .py-bestanden)

Daarnaast kies je **minstens twee** van de volgende extra technieken en gebruikt ze op een **zinvolle** manier:

- **Polymorfisme**
- **Abstract Base Classes (ABC)**
- **Iterators**
- **Generators**

Voeg een korte README of comment in `main.py` toe waarin je aangeeft:

- Welke extra technieken je gekozen hebt (bv. ABC + iterator),
- Waar in de code je die gebruikt.

Systeemvereisten

Je systeem zal bestaan uit verschillende klassen. De hieronder gegeven klassennamen zijn suggesties; je mag andere namen gebruiken, zolang de **functionaliteit** duidelijk aanwezig is.

1. Materialen in de bibliotheek

Je maakt een klassehiërarchie voor materialen.

1.1. Basisklasse voor materialen

Definieer een (eventueel abstracte) klasse `LibraryItem` met minstens:

- `title` (str)
- `item_id` (unieke identificatie, bv. string of int)
- `year` (int, publicatiejaar)

- Een manier om bij te houden of het item momenteel **uitgeleend** is of niet (bv. een boolean attribuut, of via een methode/property).

Voorzie minstens:

- Een `__str__`-methode met een leesbare beschrijving (titel, jaar, type).
- Een `__repr__`-methode die nuttig is voor debugging (korte, informatieve string).
- Een methode `mark_borrowed()` en `mark_returned()` om de uitleenstatus te beheren.

1.2. Subklassen van LibraryItem (inheritance & polymorfisme)

Maak minstens **drie** concrete subklassen, bijvoorbeeld:

- `Book(LibraryItem)`
  - extra attributen: `author`, `isbn`
- `Magazine(LibraryItem)`
  - extra attribuut: `issue_number`
- `Dvd(LibraryItem)`
  - extra attribuut: `duration_minutes`

Elke subklasse mag eigen gedrag hebben, bijvoorbeeld:

- Een methode `get_loan_period_days()` die per type een andere maximale uitleentermijn teruggeeft (bv. boek = 21 dagen, dvd = 7 dagen, tijdschrift = 14 dagen).

Dit is een natuurlijke plaats om **polymorfisme** te tonen: je werkt met `LibraryItem`-referenties, maar de concrete subklasse bepaalt de duur.

Als je met **Abstract Base Classes (ABC)** werkt:

- Maak `LibraryItem` een abstracte klasse met minstens één `@abstractmethod`, bijvoorbeeld `get_loan_period_days()`.

2. Leden en medewerkers

Je definieert klassen voor personen die de bibliotheek gebruiken.

2.1. Basisklasse Person

Maak een klasse Person met:

- name
- email

Voorzie een __str__-methode die een duidelijke tekst teruggeeft (naam + eventueel rol of e-mailadres).

2.2. Subklassen van Person

Maak minstens de volgende subklassen:

- Member(Person)
    - eventueel extra attributen, bv. member_id, datum van inschrijving.
    - houd (indien je dat handig vindt) een lijst bij van actieve uitleningen voor dit lid (composition met Loan-objecten of referenties ernaar).
- Librarian(Person)
    - bijvoorbeeld extra attribuut: employee_id.

Optioneel kun je getters/setters (of @property) gebruiken als je bv. validatie wil op email of name.

3. Uitleningen

Maak een klasse Loan die één uitlening voorstelt:

Attributen:

- item (een LibraryItem-object)
- member (een Member-object)
- borrow_date (datum/tijd van uitlening; mag bv. een datetime.date of datetime.datetime zijn)
- due_date (vervaldatum; mag berekend worden op basis van get_loan_period_days() van het item)
- optioneel: return_date (datum waarop het item wordt teruggebracht)

Methods:

- Een methode om te registreren dat een item is teruggebracht (zet return_date en markeer het item als niet meer uitgeleend).
- Een methode is_overdue(current_date) die teruggeeft of de uitlening **te laat** is.

Voorzie een zinvolle __str__-methode, bv.:

"Boek 'Titel' uitgeleend aan Jan Janssens tot 15/03/2026"

4. Bibliotheekbeheer (Library, composition & iterator/generator)

Waar composition in dit systeem zinvol is:

- De Library **bevat** items, leden en leningen → dit is een klassiek voorbeeld van composition.
- Een Member kan (optioneel) een lijst van eigen leningen bevatten → het lid **heeft** leningen.
- Een Loan bevat verwijzingen naar een LibraryItem en een Member → de uitlening **bestaat uit** deze objecten.

Deze natuurlijke relaties vormen samen de belangrijkste toepassingen van composition in dit systeem.

Maak een klasse Library die: (Library, composition & iterator/generator)

Maak een klasse Library die:

- Een collectie van LibraryItem-objecten bijhoudt.
- Een collectie van Member-objecten bijhoudt.
- Een collectie van Loan-objecten bijhoudt.

Voorzie minstens de volgende functionaliteiten:

1. add_item(item: LibraryItem)
    - Voegt een item toe aan de collectie.
2. add_member(member: Member)
    - Voegt een member toe aan de collectie.
3. borrow_item(item_id, member_id, borrow_date)
    - Zoekt het juiste item en member (op basis van id's).
    - Controleert of het item niet al uitgeleend is.

    - Berekent due_date op basis van get_loan_period_days() van het item.
    - Maakt een nieuw Loan-object aan en bewaart dit.
    - Markeert het item als uitgeleend.
4. return_item(item_id, return_date)
    - Zoekt de bijbehorende Loan.
    - Registreert de terugbrengst (return_date).
    - Markeert het item als beschikbaar.

Magic methods & iterators

Implementeer minimaal:

- Library.__len__() → geeft het aantal items in de bibliotheek terug.

Kies daarnaast **minstens één** van deze opties:

- Maak Library zelf **iterable** (via __iter__), en laat iteratie lopen over alle items in de bibliotheek.
  Voorbeeldgebruik:

  ```
  for item in library:
      print(item)
  ```

- Maak een aparte klasse LoanRegistry die alle leningen bevat en een **iterator** is over actieve leningen (__iter__, __next__).

- Schrijf een **generatorfunctie**, bv. overdue_loans(library, current_date) die alle leningen oplevert waarvoor is_overdue(current_date) True is.

Je mag meer dan één van deze opties combineren, maar minstens één is verplicht.

5. Fouten & custom exceptions

Voorzie minstens één eigen exceptieklasse, bijvoorbeeld:

- LibraryError(Exception) als algemene basis.

En/of meer specifieke excepties zoals:

- ItemNotAvailableError(LibraryError) als iemand een item probeert te lenen dat al uitgeleend is.
- MemberNotFoundError(LibraryError) als een member_id niet bestaat.
- ItemNotFoundError(LibraryError) als een item_id niet bestaat.

Gebruik deze excepties in je methoden (borrow_item, return_item, ...) en vang ze op in je hoofdprogramma (main.py) met try/except-blokken. Toon duidelijke foutboodschappen aan de gebruiker (in de console).

6. Modules & structuur

Splits je project op in **minstens twee modules**, bijvoorbeeld:

- items.py – klassen rond LibraryItem, Book, Magazine, Dvd, ...
- people.py – klassen rond Person, Member, Librarian, ...
- loans.py – Loan (en eventueel LoanRegistry)
- library.py – de Library-klasse
- main.py – waar je het systeem demonstreert

Minimaal verwachten we:

- Eén module met de modellen/klassen (bv. models.py of opgesplitst), en
- Eén module main.py waarin je de werking demonstreert.

In main.py toon je minstens:

1. Het aanmaken van een aantal items (boeken, tijdschriften, dvd's).
2. Het aanmaken van enkele members (en eventueel een librarian).
3. Enkele uitleningen en terugbrengacties.
4. De foutafhandeling: bijvoorbeeld proberen een al uitgeleend item opnieuw te lenen, of een niet-bestaand item lenen.
5. Het gebruik van je iterator of generator (afhankelijk van wat je gekozen hebt).
6. Enkele print()-statements die gebruikmaken van je __str__-implementaties.